

- 1. Why are we focusing on a single round of Pong during the training phase? In other words, suppose the score is 0:0 vs 10:0 in your favor (and suppose in both cases you win 21:0). What values will your Q network learn for the starting state in each of those two cases?**

Our state representation does not include the score. Therefore, the network cannot distinguish between the start of a game (0:0) and the middle of a game (10:0). If we trained on full games, the network would receive conflicting signals for the exact same input state. At 0:0, the future return is +21 (assuming you win all remaining points). At 10:0, The future return is only +11. The result of this is that the Q-network would try to converge on two vastly different values (21 vs. 11) for the same input, causing unstable learning. By training on single rounds, the target return is always ± 1 regardless of the broader game context. This ensures the value function is consistent and well-defined for the observed state.

- 2. What changes did you have to make in Pong in addition to your Cartpole approach?**

While the DDQN algorithm remained identical, the interface between the agent and the environment required key adaptations:

- 1) State Representation from Images:
 - Cartpole: Used the raw environment vector directly.
 - Pong: The environment returns RGB images. `get_state_from_image` distills these frames into a compact 4D vector (paddle Y-coords and ball X/Y coords) so the network learns from physics states, not pixels.
- 2) Frame Skipping:
 - Cartpole: Actions were taken every single timestep.
 - Pong: Used the provided `env_step(env, action, frame_skip)` helper, which repeats the same action for several frames (`frame_skip = 4`) and accumulates the reward, breaking early if a point is scored or the round ends. This reduces redundancy in frames and speeds up learning.
- 3) Training Termination:
 - Cartpole: Episodes ended only on failure (pole drop).
 - Pong: Episodes ended after one round, i.e., when `reward != 0` (the agent or opponent scores a point) or when `done=True`.
- 4) Replay buffer dimensionality and initialization
 - Cartpole: Instantiated the replay buffer using `state_dim = env.observation_space.shape[0]`.
 - Pong: Used `state_dim = 4` (the 4D extracted state) and stored those state vectors instead of raw images, but still used the same `ReplayBuffer` class.
- 5) Evaluation logic for full games to 21
 - Cartpole: `eval_policy` runs standard episodes and averages the total return over 10 runs.
 - For Pong's `eval_policy`, I needed to use the Pong-specific step function and state extraction (`env_step + get_state_from_image`) and run each episode until a full game to 21 is completed, accumulating the per-point rewards. The evaluation then reports the average undiscounted score over 10 full games.

Hyperparameters (like number of steps, buffer size, etc.) are similar between the two, but Pong uses a longer budget (200k steps) and can be more sensitive to choices like learning rate, which I tuned slightly but kept the core DDQN structure identical.